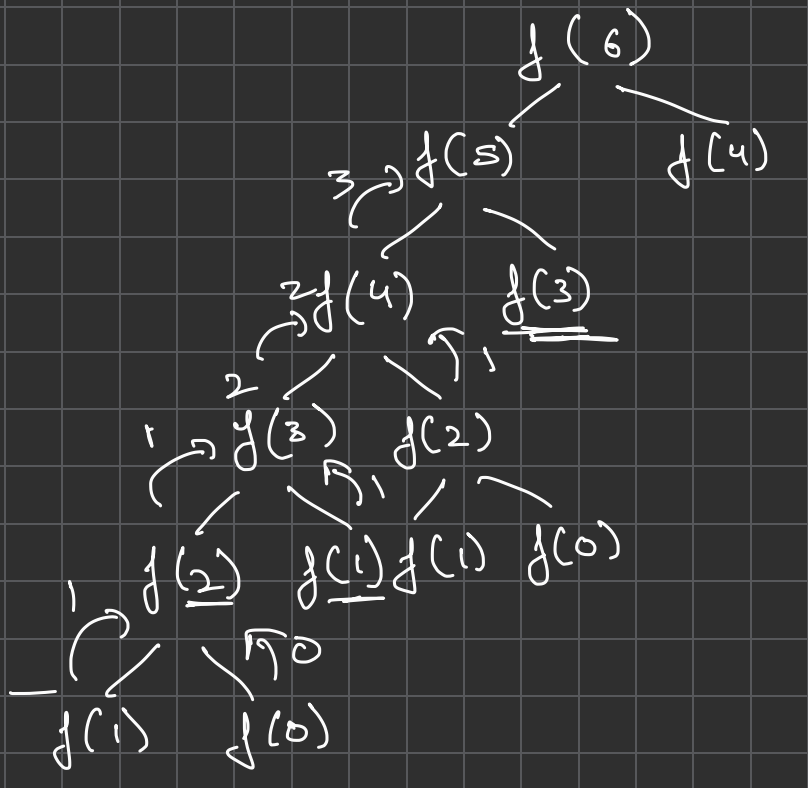
Recursion & Trees

⇒ A programming & mathematical technique where a function calls itself or indirectly to solve a problem by breaking it down into smaller, similar subproblems.

indices: 0 1 2 3 4 5 6 7

fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, ...




```
f(n)
{
  if (n==0): ret 0
  if (n==1): ret 1
  return f(n-1) +
         f(n-2)
}
```

PROBLEM - 1

78. Subsets

Medium | Topics | Companies

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

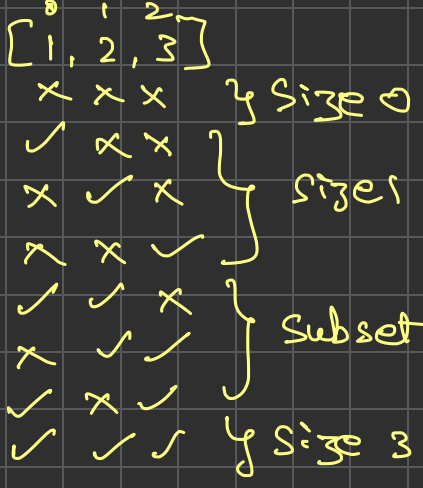
- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- All the numbers of `nums` are **unique**.

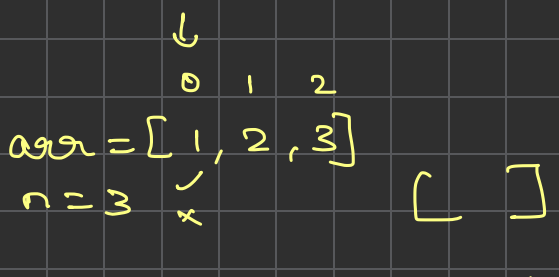
arr = [1, 2, 3]

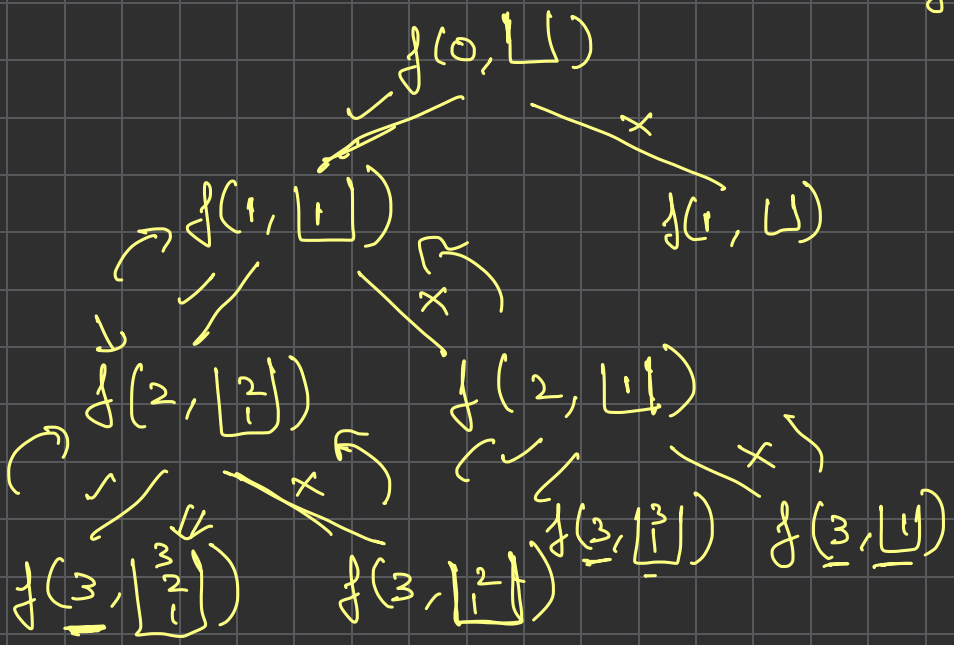
output

[[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]

pick, not-pick technique





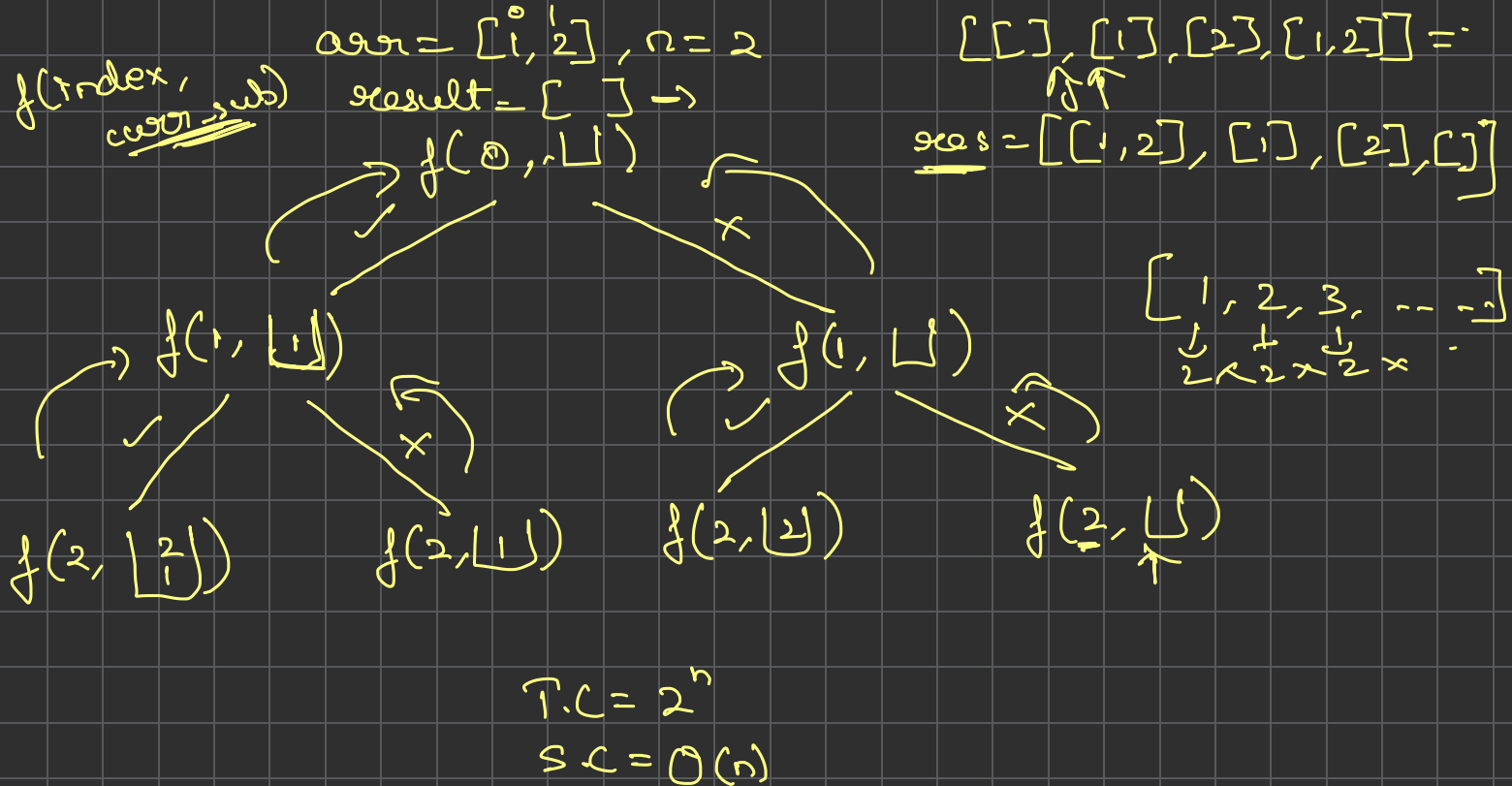


[[1,2,3], [1,2], [1,3], [1], ----]

pseudocode:

```
result = []
f(index, sub)
{ if (index > (n-1))
      result.append(subset)

  #pick
  sub.append(arr[index])
  f(index+1, sub)
  sub.pop() -
  # not pick
  f(index+1, sub)
```


arr = [1, 2], n = 2
result = [ ] →
f(index, curr, ub)
f(0, 1)
f(1, 1)
f(1, 1)
f(2, 2)
f(2, 1)
f(2, 2)
f(2, 1)
[ ], [1], [2], [1,2] =
res = [[1,2], [1], [2], []]
[1, 2, 3, ...]
1/2 × 2 × 1/2 ×

$T.C = 2^n$

$S.C = O(n)$

PROBLEM-2

39. Combination Sum

Medium | Topics | Companies

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all* ***unique combinations*** *of* `candidates` *where the chosen numbers sum to* `target`. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

Example 1:

```
Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]
Explanation:
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7.
These are the only two combinations.
```

Example 2:

```
Input: candidates = [2,3,5], target = 8
Output: [[2,2,2,2],[2,3,3],[3,5]]
```

Example 3:

```
Input: candidates = [2], target = 1
Output: []
```

candidates = [2, 3, 6, 7] (0 1 2 3), target = 7

[7] == target ✓

sum([2,2,3]) == target ✓

[2,3,5], target = 8

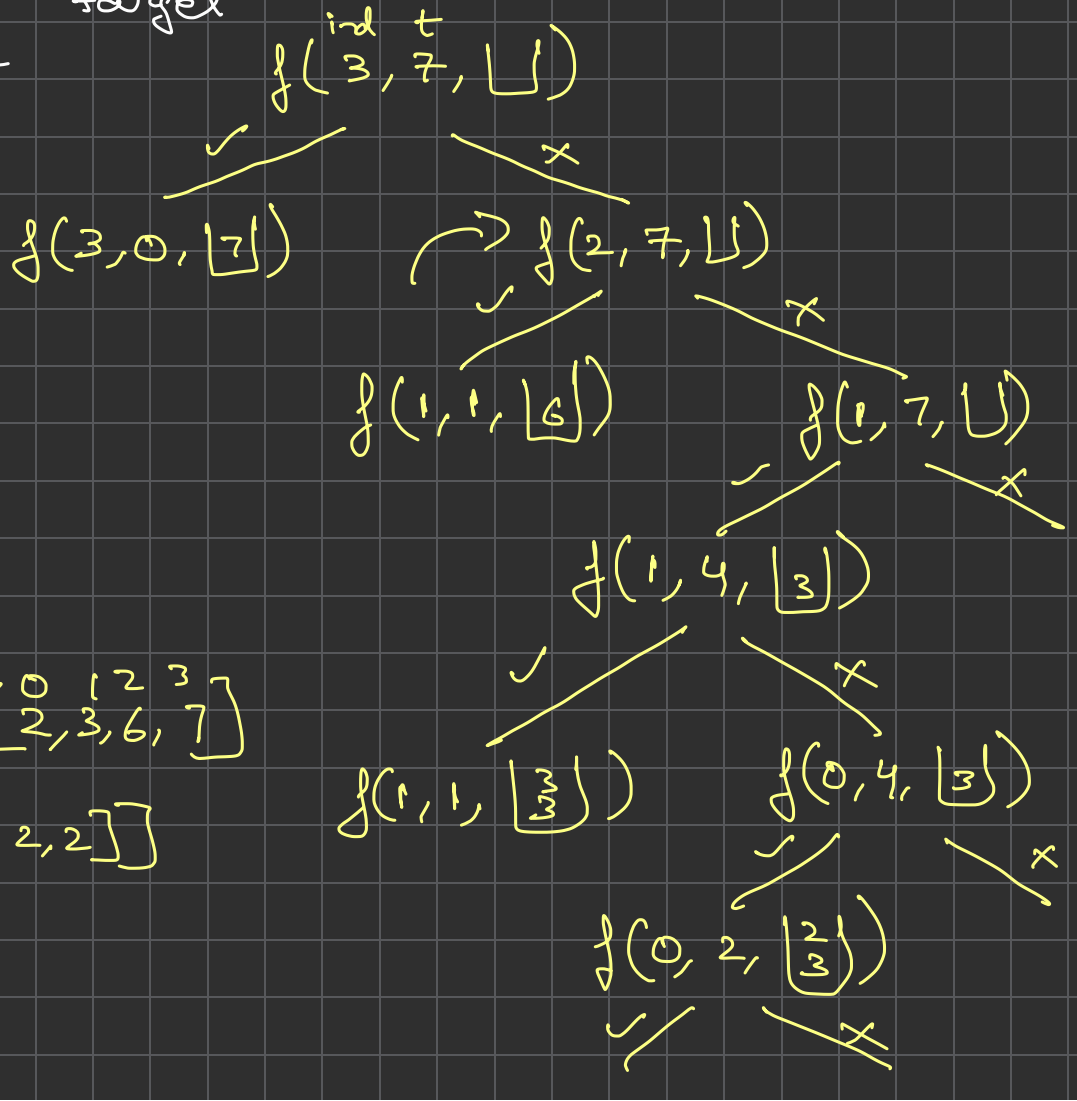
sum([2,2,2,2]) == target

sum([2,3,3]) == target

sum([3,5]) == target

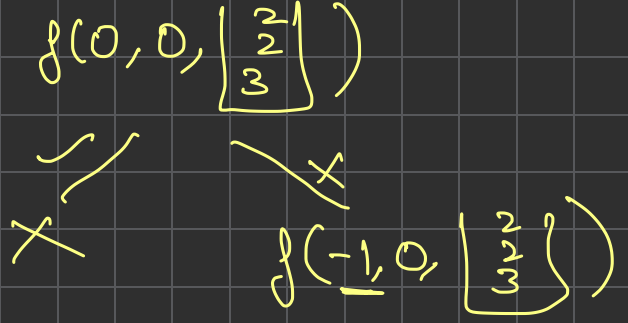
f(n, target) } generate all subsets from arr of size n, with subsets sum as target

[2, 3, 6, 7] (0 1 2 3), t = 7



[2, 3, 6, 7] (0 1 2 3)

result = [[7], [3, 2, 2]]



pseudocode:

```
f(index, target, sub)
{
    # base case
    if (index<0)
        if (target==0)
            result.append(sub)
    if (nums[index] <= target){
    # pick & stay
    f(index, target - nums[index], sub + [nums[index]])
    }

    # not pick
    f(index-1, target, sub)

}
```

T.C = $\sum K \cdot K \approx 2^n$

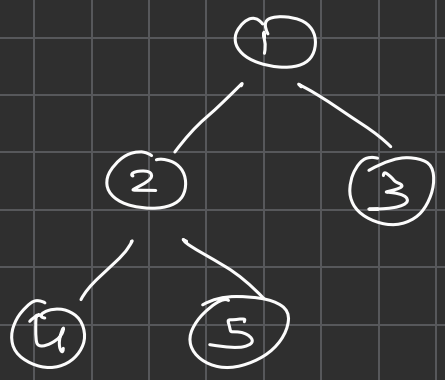
S.C = $K = n$

TREES:

DFS & BFS / level order.

T.C = O(n)
S.C = O(n)

// 1.) Pre-order (Root, left, right)
// 2.) In-order (left, root, right)
// 3.) post-order (left, right, root)



pre-order: [1, 2, 4, 5, 3]
In-order: [4, 2, 5, 1, 3]
post-order: [4, 5, 2, 3, 1]

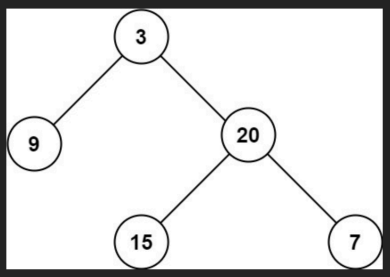
104. Maximum Depth of Binary Tree

Easy | Topics | Companies

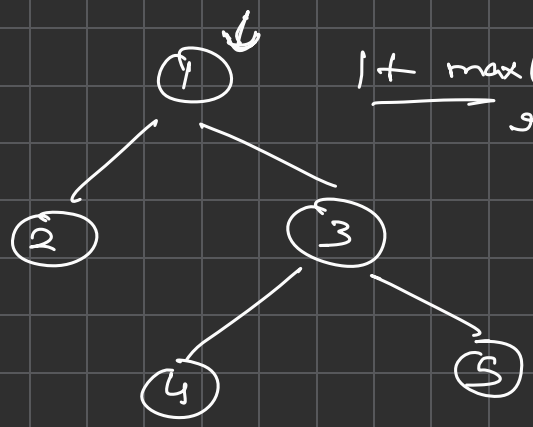
Given the `root` of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

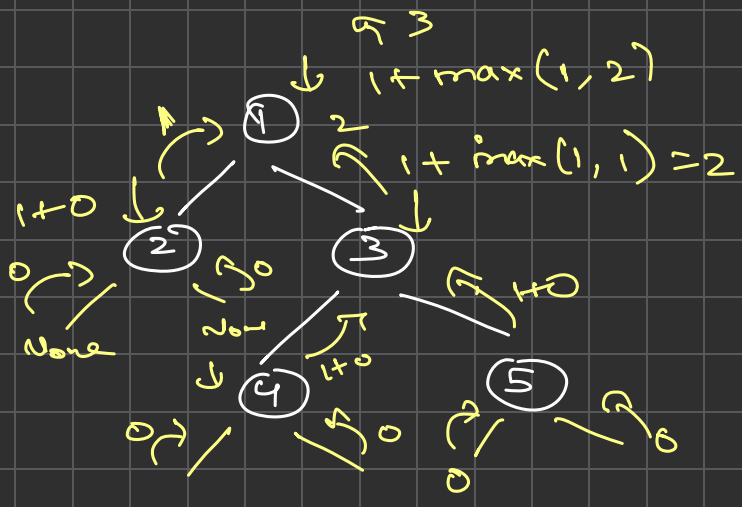
Example 1:

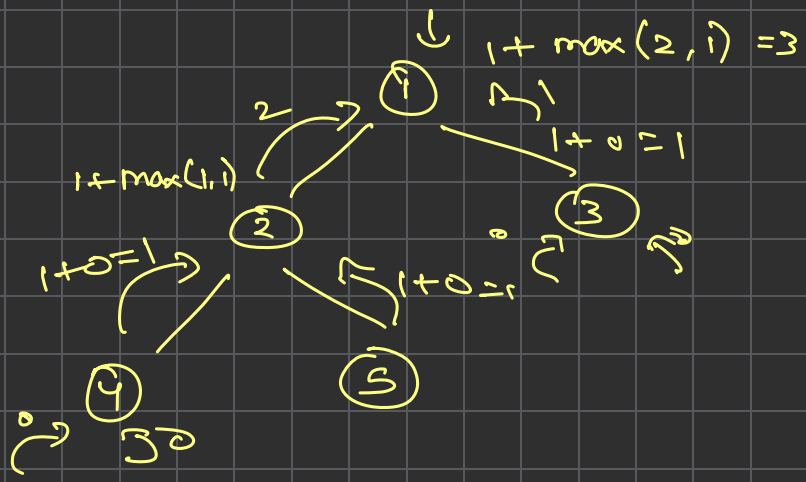



```
Input: root = [3,9,20,null,null,15,7]
Output: 3
```




```
f(root)
{ if(root==None)
    return 0
  ld = f(root.left)
  rd = f(root.right)
  return 1+max(ld,rd)
}
```





T.C = O(n)

S.C = O(n)

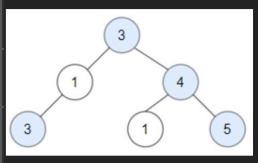
1448. Count Good Nodes in Binary Tree

Medium | Topics | Companies | Hint

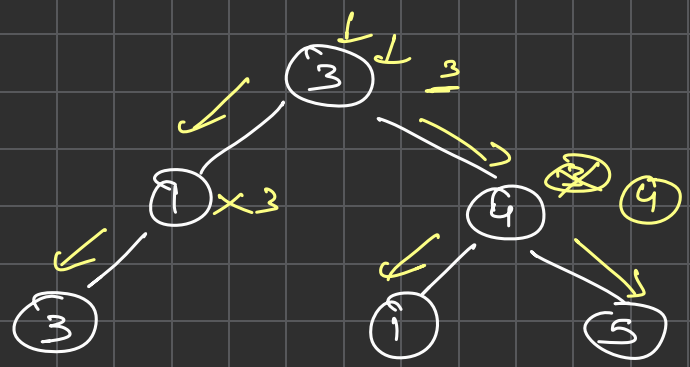
Given a binary tree `root`, a node *X* in the tree is named **good** if in the path from root to *X* there are no nodes with a value *greater than* X.

Return the number of **good** nodes in the binary tree.

Example 1:




```
Input: root = [3,1,4,3,null,1,5]
Output: 4
Explanation: Nodes in blue are good.
Root Node (3) is always a good node.
Node 4 -> (3,4) is the maximum value in the path starting from the root.
Node 5 -> (3,4,5) is the maximum value in the path
Node 3 -> (3,1,3) is the maximum value in the path.
```



count = 4

T.C = O(n)

S.C = O(n)

pseudocode:

```
f(root, max_so_far)
{
  if root == None:
      return 0
  if root.val >= max_so_far:
      max_so_far = root.val
      return 1 + f(root.left)
               + f(root.right)
  return f(root.left)
         + f(root.right)
}
```